



Força Bruta e Busca Exaustiva

WALLESSON CAVALCANTE DA SILVA

PROF. DR. CRISTON PEREIRA DE SOUZA

Força Bruta

- Algoritmo simples e direto de acordo com a definição do problema;
- Não é aplicado esforço mental para projetar um algoritmo eficiente;
- Computador realiza esforço de resolver o problema de forma menos eficiente;
- Geralmente os problemas podem ser resolvidos de forma mais eficiente que o algoritmo de força bruta;
- Raramente produz um algoritmo eficiente.

Força Bruta

Exemplo

- Calcular b^n
 - $b^n = b * b * \dots * b$.
- Considerando a multiplicação com tempo constante, o algoritmo será $O(n)$
- Algoritmo mais eficiente:
 - $b^n = b^{\lfloor \frac{n}{2} \rfloor} * b^{\lfloor \frac{n}{2} \rfloor}$ se n é par
 - $b^n = b^{\lfloor \frac{n}{2} \rfloor} * b^{\lfloor \frac{n}{2} \rfloor} * b$ se n é ímpar
 - $T(n) = T\left(\frac{n}{2}\right) + \theta(1) \in \theta(\log n)$
- Problema de ordenação
 - Bolha ou ordenação por seleção – $O(n^2)$
 - Merge Sort - $O(n \log n)$

Força Bruta

Importância do projeto de força bruta:

- É mais **provável que o problema admita solução por força bruta**. Isto não é verdade para as outras técnicas de projeto;
- Para **alguns problemas** importantes produz algoritmos com **eficiência satisfatória**, como no problema de ordenação;
- O **esforço adicional** para produzir algoritmo mais eficiente **pode ser desnecessário** quando o algoritmo for usado poucas vezes e com tamanhos de entrada que produzem tempo de execução aceitável;
- Pode servir como **ponto de partida** para soluções mais sofisticadas;
- Pode **ajudar a testar** outros algoritmos, pois conseguimos executar e produzir soluções corretas para instâncias pequenas.

Busca Exaustiva

- Abordagem de força bruta para um problema combinatório;
- Objetivo é gerar todos os objetos do domínio (satisfazendo as restrições) em busca do elemento que satisfaz a propriedade desejada (por exemplo maximizar/minimizar uma função objetivo);
- Crescimento exponencial do domínio de objetos combinatórios faz com que a abordagem seja viável somente para entradas pequenas.

Busca Exaustiva

Exemplo

- Encontrar caminho de comprimento mínimo entre dois nós s e t em um grafo direcionado com pesos não negativos nas arestas.
- Combinatório: um caminho é um subconjunto das arestas
- Otimização: deseja-se encontrar o caminho cuja soma dos pesos das arestas seja o mínimo.
- Solução: Dijkstra
- Solução busca exaustiva:
 - Gera-se todo subconjunto A de arestas
 - Testa se A produz um caminho de s até t .
 - Se A possui um caminho, e o custo é o menor dentre os caminhos avaliados, guardamos a solução de A .
 - Ao final é retornada a melhor solução encontrada.

Busca Exaustiva

- Geração de permutações
 - $L[1...n] = n!$ permutações
 - Ordem lexicográfica
 - A permutação a precede a permutação b se para algum $1 \leq k \leq n$ temos que $a[1] = b[1], a[2] = b[2], \dots, a[k-1] = b[k-1]$ e $a[k] < b[k]$. 23415 precede 23514.
 - Gerando todas as permutações em ordem lexicográfica dada uma permutação a , onde a não é a última permutação.

-
- `proxima_permutacao`
 - Passo 1: Encontre o índice $1 \leq j < n$ tal que $a[j] < a[j+1]$ (o último valor menor que o sucessor)
 - Passo 2: Encontre o maior índice k tal que $j < k \leq n$ e $a[j] < a[k]$. Ou seja, $a[k]$ é o menor elemento dentre os que estão à direita de $a[j]$ e são maiores que $a[j]$.
 - Passo 3: Troque $a[j]$ por $a[k]$
 - Passo 4: Coloque os elementos em $a[j+1..n]$ em ordem crescente.
 - Todas as permutações $O(n*n!)$

Exemplo

- Qual o sucessor da permutação 362541?

índice	1	2	3	4	5	6
Passo 1:	3	6	2	5	4	1
Passo 2:	3	6	2	5	4	1
Passo 3:	3	6	4	5	2	1
Passo 4:	3	6	4	1	2	5

- Resposta: 364125

Algoritmo: próxima_permutação($a[1..n]$)

Entrada: Lista $a[1..n]$ contendo uma permutação de $\{1, 2, \dots, n\}$.

A permutação a não é a última ($[n, n - 1, \dots, 2, 1]$).

Saída : Modificações na permutação $a[1..n]$ de modo a torná-la a próxima na ordem lexicográfica.

// Passo 1: j é o maior índice tal que $a[j] < a[j + 1]$

```
1  $j \leftarrow n - 1$   
2 while  $a[j] > a[j + 1]$  do  
3    $j \leftarrow j - 1$ 
```

// Passo 2: $a[k]$ é o menor elemento maior que $a[j]$ à direita de $a[j]$

```
4  $k \leftarrow n$   
5 while  $a[j] > a[k]$  do  
6    $k \leftarrow k - 1$ 
```

// Passo 3: troca $a[j]$ com $a[k]$

```
7  $(a[j], a[k]) \leftarrow (a[k], a[j])$ 
```

// Passo 4: coloca elementos à direita de $a[j]$ em ordem crescente

```
8  $(s, r) \leftarrow (j + 1, n)$   
9 while  $s < r$  do  
10    $(a[s], a[r]) \leftarrow (a[r], a[s])$  // troca  $a[s]$  com  $a[r]$   
11    $(s, r) \leftarrow (s + 1, r - 1)$ 
```

Busca Exaustiva

Geração de subconjuntos

- $O(2^n)$
- Existência de uma bijeção entre subconjuntos e sequências de bits.
 - $A = \{a, b, c, d, e, f, g, h\}$ 0 1 0 0 1 0 1 1 --> subconjunto {b,e,g,h}
 - $a = \{b, e, g, h\}$ a b c d e f g h
- proxima_sequencia_bits
 - Passo 1: Da direita para a esquerda, enquanto encontrar bit 1 troque por 0
 - Passo 2: Troque por 1 o primeiro bit 0 encontrado no passo anterior.
- Complexidade $O(n)$, logo para gerar todos os subconjuntos a complexidade é $O(n \cdot 2^n)$

Algoritmo: próxima_sequência_bits($b[0..n - 1]$)

Entrada: Lista $b[0..n - 1]$ contendo uma sequência de bits
 $b[n - 1]b[n - 2] \cdots b[1]b[0]$.

A sequência de bits b não é a última (11...11).

Saída : Modificações na sequência de bits $b[0..n - 1]$ de modo a torná-la a representação do próximo número inteiro.

```
1  $i \leftarrow 0$ 
2 while  $b[i] = 1$  do
3    $b[i] \leftarrow 0$ 
4    $i \leftarrow i + 1$ 
5  $b[i] \leftarrow 1$ 
```

Exemplo

0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1

Busca Exaustiva

Geração de r -combinações

- Dado um conjunto $\{1, 2, \dots, n\}$, gerar todos os subconjuntos de tamanho r .
- Representação dos valores em lista e armazenados em ordem crescente para facilitar.
- Uma r -combinação $a[1\dots r]$ em ordem lexicográfica pode ser obtida:
- Primeiro assumimos que a não é a última r -combinação, ou seja $u=[n-r+1, n-r+2, \dots, n]$ $a \neq u$. u é composto pelos r maiores valores da lista.
 - $A [1, 2, 3, 4, 5, 6, 7]$
 - 4-combinações
 - $u[4, 5, 6, 7]$

Busca Exaustiva

proxima_r_combinação

- Passo 1: localizar o maior i tal que $a[i]$ que difere de $u[i]$ (valor da posição correspondente da última r -combinação). Ou seja, desejamos determinar o maior valor $1 \leq i \leq r$ tal que $a[i] \neq n-r+i$.
- Passo 2: Incremente $a[i]$
- Passo 3: Coloque os sucessores de $a[i]$ nas posições maiores que i . Ou seja, para cada j de $i+1$ até r , atribua para $a[j]$ o valor de $a[i] + j - i$.

Exemplo

- $A [1, 2, 3, 4, 5, 6, 7]$
- 4-combinações
- $u = [4, 5, 6, 7]$
- $a = [1, 3, 6, 7]$ qual é a próxima 4-combinação?

- Passo 1: localizar o maior i tal que $a[i]$ que difere de $u[i]$
- Passo 2: Incremente $a[i]$
- Passo 3: para cada j de $i + 1$ até r , atribua para $a[j]$ o valor de $a[i] + j - i$.

- Resposta: $[1, 4, 5, 6]$

índice	1	2	3	4
a	1	3	6	7
u	4	5	6	7
índice	1	2	3	4
a	1	4	5	6
u	4	5	6	7

Algoritmo: próxima_r_combinação($n, a[1..r]$)

Entrada: Lista $a[1..r]$ contendo uma r -combinação de $\{1, 2, \dots, n\}$ em ordem crescente ($a[1] < a[2] < \dots < a[r]$).

A r -combinação a não é a última ($u = [n - r + 1, \dots, n]$).

Saída : Modificações na lista $a[1..r]$ de modo a torná-la a próxima na ordem lexicográfica, e com elementos em ordem crescente.

// Passo 1: localiza maior i tal que $a[i]$ que difere de $u[i]$

```
1  $i \leftarrow r$ 
2 while  $a[i] = n - r + i$  do
3    $i \leftarrow i - 1$ 
```

// Passo 2: incrementa $a[i]$

```
4  $a[i] \leftarrow a[i] + 1$ 
```

// Passo 3: coloca os sucessores de $a[i]$ nas posições maiores que i

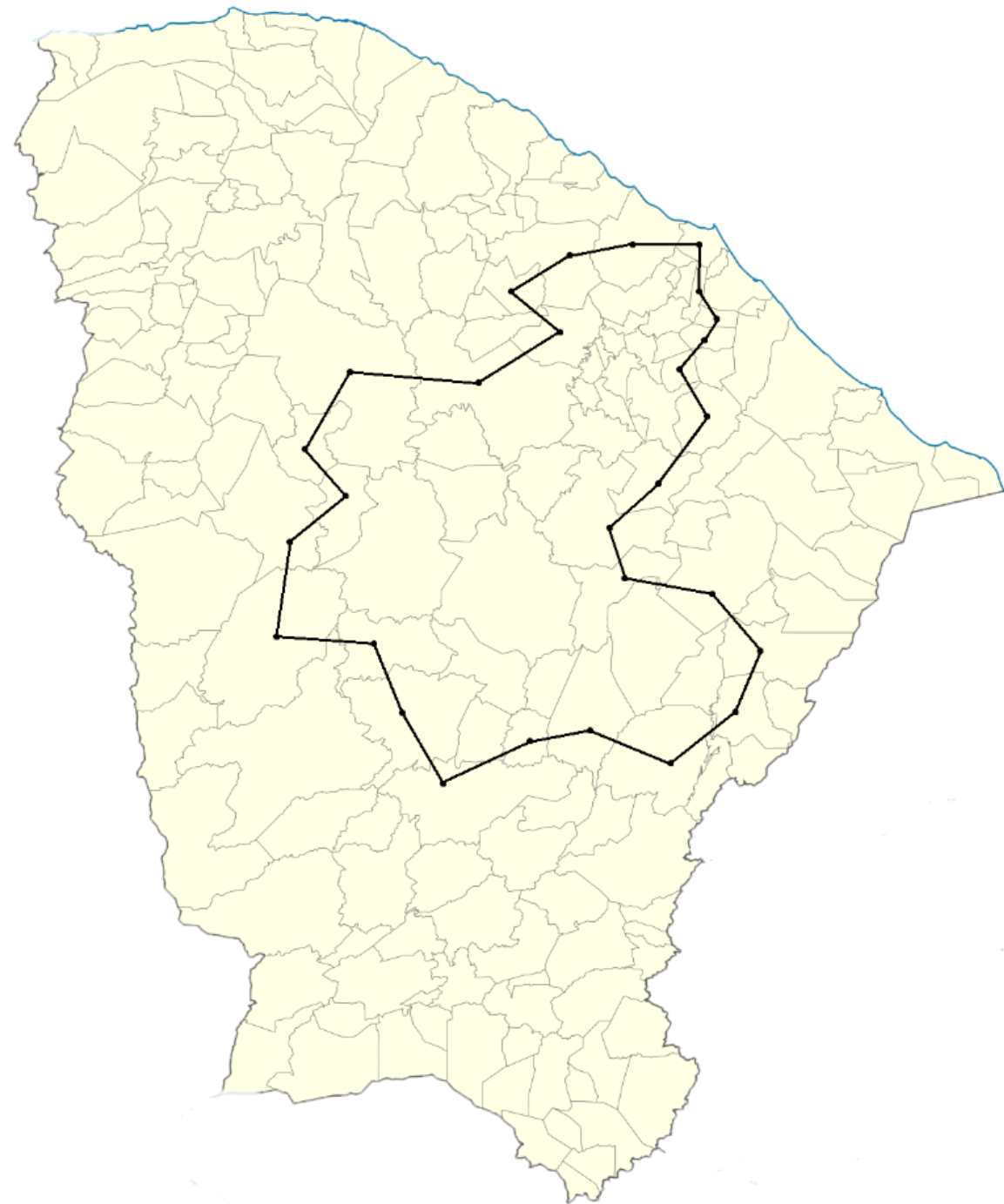
```
5 for  $j$  de  $i + 1$  até  $r$  do
6    $a[j] \leftarrow a[i] + j - i$ 
```

Problema do Caixeiro Viajante

No problema do caixeiro viajantes temos um grafo não direcionado com pesos não negativos $d[u][v]$ em cada arestas (u, v) , e desejamos **determinar o percurso com menor peso total que passa por todos os vértices e retorna para o vértice inicial**. Por exemplo, suponha que cada vértice representa uma cidade, e o peso de cada aresta (u, v) representa a duração da viagem entre as cidades u e v . Neste caso a solução será um trajeto que passa por todas as cidades gastando o menor tempo total. Indexando os vértices de 1 até n , **cada percurso é uma permutação** $a[1..n]$ de $\{1, 2, \dots, n\}$. O **custo da permutação** $a[1..n]$ vale:

$$\text{custo}(a) = d[a[n]][a[1]] + \sum_{j=2}^n d[a[j-1]][a[j]]$$

Para resolver por **busca exaustiva** basta **gerar todas as permutações** das n cidades. Para **reduzir as simetrias** (soluções idênticas), podemos fixar que a primeira cidade sempre vale 1, ou seja, $a[1] = 1$. Neste caso basta iniciar na permutação $[1, 2, \dots, n]$ e explorar as próximas $(n - 1)!$ permutações na ordem lexicográfica.



Algoritmo: caixeiro_viajante($d[1..n][1..n]$)

Entrada: Cada elemento $d[i][j]$ da matriz d fornece a distância da cidade i para a cidade j . As distâncias são reais não negativos.

Saída : Retorna permutação das cidades de custo mínimo.

```
1  $a[1..n] \leftarrow [1, 2, \dots, n]$            // (1a permutação das  $n$  cidades)       $O(n)$ 
2 for  $i$  de 1 até  $(n - 1)!$  do                //  $O((n - 1)!)$ 
    // calcula o custo da permutação  $a$ 
3     custo  $\leftarrow d[a[n]][a[1]]$            //  $O((n - 1)!)$ 
4     for  $j$  de 2 até  $n$  do                    //  $O(n(n - 1)!) = O(n!)$ 
5         custo  $\leftarrow$  custo +  $d[a[j - 1]][a[j]]$  //  $O(n!)$ 

    // armazena  $a$  se tiver o menor custo
6     if custo < menor_custo then              //  $O((n - 1)!)$ 
7         menor_custo  $\leftarrow$  custo          //  $O((n - 1)!)$ 
8         solução  $\leftarrow a$                  //  $O((n - 1)!)$ 
9     próxima_permutacao( $a$ )                   //  $O(n(n - 1)!) = O(n!)$ 
10 return solução                             //  $O(1)$ 
```

